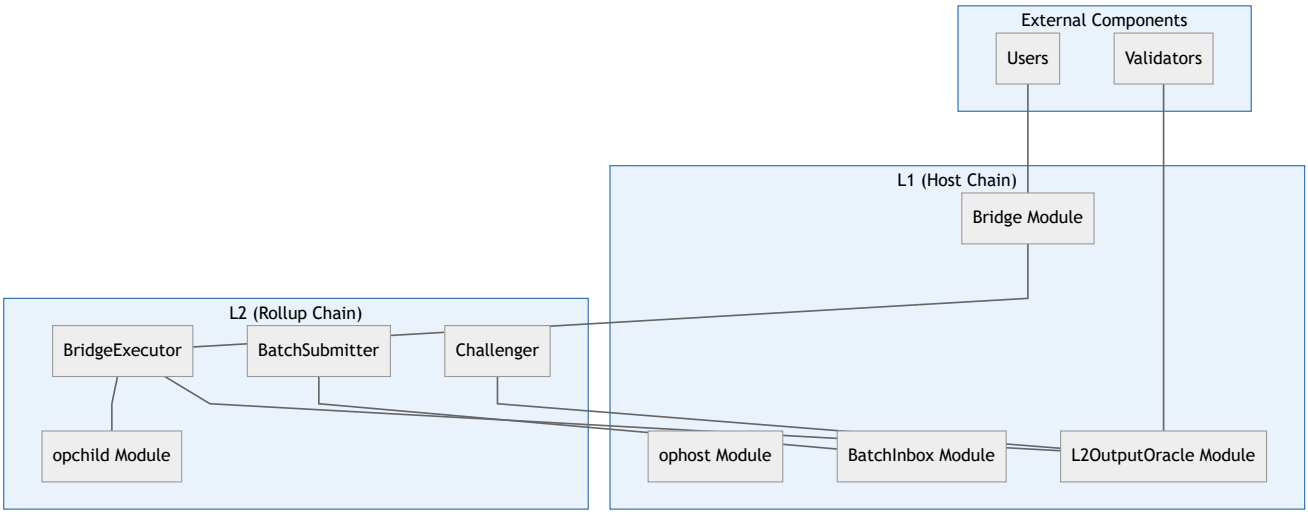
OPinit 전체 아키텍처 개요

소개

OPinit은 CosmosSDK 기반의 VM-agnostic Optimistic Rollup 프레임워크입니다. Optimism의 Bedrock 인터페이스와 유사한 구조를 가지며, Initia L1 거버넌스 모델을 활용하여 효율적인 fraud-proof 분쟁 해결을 제공합니다.

전체 시스템 아키텍처

OPinit은 L1(호스트 체인)과 L2(롤업 체인) 두 레이어로 구성되며, 각 레이어는 특정한 역할을 담당합니다:



L1 컴포넌트 (호스트 체인)

1. ophost 모듈

L1에서 롤업 운영을 관리하는 핵심 모듈입니다.

주요 구조체

```
type Keeper struct {
cdc codec.Codec
storeService corestoretypes.KVStoreService
msgRouter baseapp.MessageRouter
// Collections for state management
NextBridgeId collections.Sequence
BridgeConfigs collections.Map[uint64, types.BridgeConfig]
BatchInfos collections.Map[collections.Pair[uint64, uint64], types.BatchInfoWith
OutputProposals collections.Map[collections.Pair[uint64, uint64], types.Output]
ProvenWithdrawals collections.Map[collections.Pair[uint64, []byte], bool]
}
```

핵심 기능

- **브리지 관리**: `MsgCreateBridge` 를 통한 새로운 브리지 생성
- **입금 처리**: `MsgInitiateTokenDeposit` 를 통한 L1→L2 토큰 전송
- **출금 완료**: `MsgFinalizeTokenWithdrawal` 를 통한 L2→L1 토큰 전송 완료
- **Output 관리**: `MsgProposeOutput` , `MsgDeleteOutput` 를 통한 L2 상태 관리

2. Bridge Module

L1과 L2 간 토큰 전송을 담당하는 모듈입니다.

주요 인터페이스

- `initialize_deposit` : 사용자가 L1에서 L2로 토큰을 전송할 때 사용
- `finalize_withdrawal` : 브리지 실행자가 L2에서 L1으로의 출금을 완료할 때 사용

3. BatchInbox Module

데이터 가용성(DA) 레이어 역할을 하는 모듈입니다.

특징

- Celestia 등 다른 DA 솔루션으로 교체 가능
- 가스 비용 절약을 위해 임의의 바이트를 인수로 받는 빈 함수 인터페이스만 노출
- L2 데이터가 상태가 아닌 tx db(calldata)에 저장되도록 보장

4. L2OutputOracle Module

L2 output root를 저장하여 블록 finalization을 담당하는 모듈입니다.

핵심 기능

- L2 output root 저장 및 관리
- 출금 트랜잭션 증명을 위한 output root 제공
- Challenger의 모니터링 및 챌린지 처리

L2 컴포넌트 (롤업 체인)

1. opchild 모듈

L2에서 롤업 특화 기능을 제공하는 핵심 모듈입니다.

주요 구조체

```
type Keeper struct {
cdc codec.Codec
storeService corestoretypes.KVStoreService
// Sequence management
NextL1Sequence collections.Sequence
NextL2Sequence collections.Sequence
// Bridge and validator management
BridgeInfo collections.Item[types.BridgeInfo]
Validators collections.Map[[]byte, types.Validator]
ValidatorsByConsAddr collections.Map[[]byte, []byte]
// Token and migration management
DenomPairs collections.Map[string, string]
MigrationInfos collections.Map[string, types.MigrationInfo]
}
```

핵심 기능

- **검증자 관리**: `MsgAddValidator` , `MsgRemoveValidator` 를 통한 권한 기반 검증자 관리
- **브리지 실행**: `MsgFinalizeTokenDeposit` , `MsgInitiateTokenWithdrawal` 를 통한 토큰 전송
- **권한 실행**: `MsgExecuteMessages` 를 통한 브리지 실행자 권한 메시지 실행

2. BridgeExecutor

L2 롤업의 핵심 컴포넌트로 다음 작업을 담당합니다:

- L1 입금 트랜잭션을 L2에서 완료
- 출금 트랜잭션 저장 Merkle Tree 구성
- L2 output root 계산
- 사용자에게 출금 증명(Merkle Proofs) 제공

3. BatchSubmitter

L1의 `BatchInbox` 모듈에 트랜잭션 배치를 제출하는 백그라운드 프로세스입니다.

4. Challenger

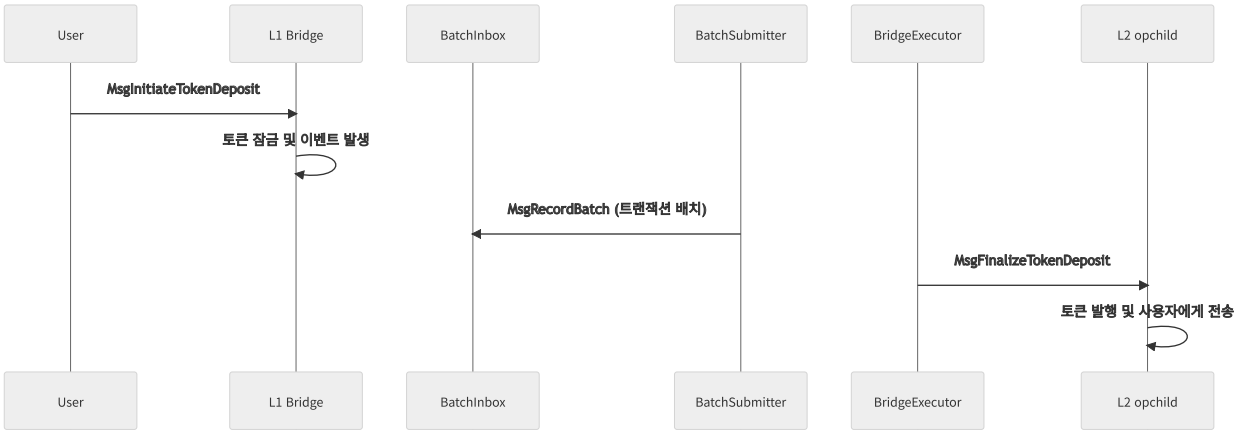
Output oracle에서 유효하지 않은 output 제안을 삭제할 수 있는 엔티티입니다.

주요 역할

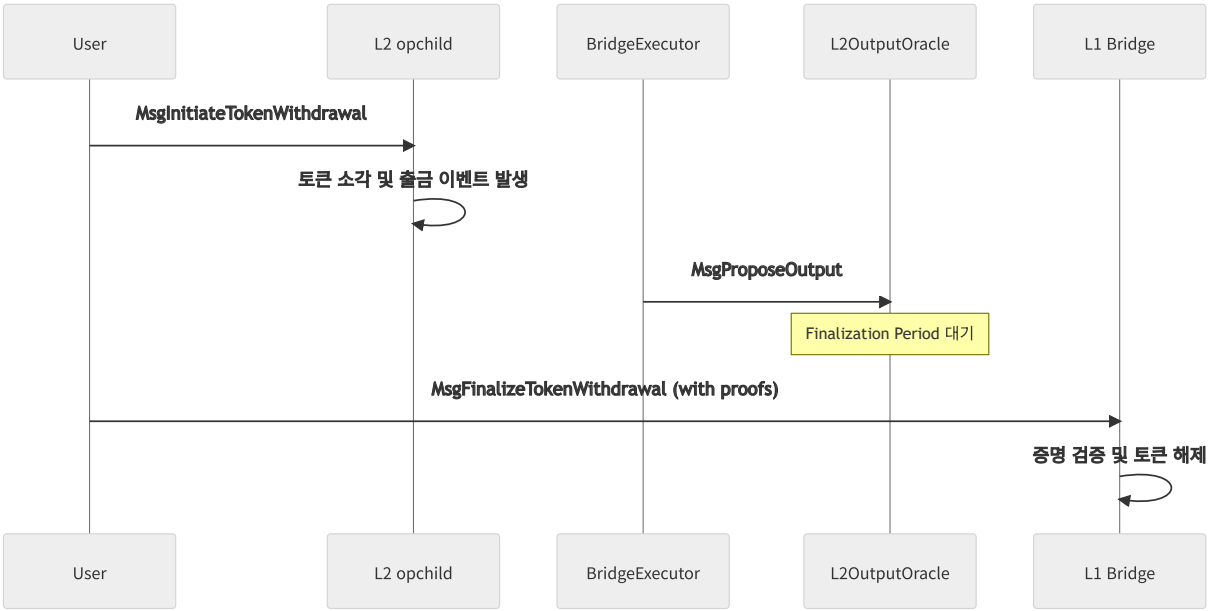
- 브리지 실행자가 수행하는 output root 생성 과정을 모방하여 유효성 검증
- L1에서 L2로의 입금 트랜잭션 모니터링으로 검열 저항성 보장
- IBC 릴레이어 운영을 통한 즉시 브리지 작업 지원

시스템 전체 데이터 플로우

L1 → L2 입금 플로우


User
L1 Bridge
BatchInbox
BatchSubmitter
BridgeExecutor
L2 opchild
MsgInitiateTokenDeposit
토큰 잠금 및 이벤트 발생
MsgRecordBatch (트랜잭션 배치)
MsgFinalizeTokenDeposit
토큰 발행 및 사용자에게 전송
User
L1 Bridge
BatchInbox
BatchSubmitter
BridgeExecutor
L2 opchild

L2 → L1 출금 플로우



분쟁 해결 프로세스

OPinit은 L1 거버넌스 보안을 활용한 단순화된 분쟁 메커니즘을 사용합니다:

분쟁 프로세스

1. **Challenger의 액션**: 유효하지 않은 output root를 output oracle 모듈에서 삭제

2. **거버넌스 제안**:

- Challenger: 유효하지 않은 output root를 계속 제출하는 proposer 주소 변경 제안

- Proposer: 유효한 output root를 계속 삭제하는 challenger 주소 변경 제안

3. **L1 검증자 결정**: L2 노드를 실행하여 L2 상태와 데이터 입력으로 유효성 결정

거버넌스 기반 보안 모델

- Cosmos의 shared security와 유사하지만 모든 검증자가 전체 L2 노드를 실행할 필요 없음
- 분쟁 발생 시에만 L1의 마지막 L2 상태로 분쟁 블록을 실행
- 전체 L2 블록이 아닌 분쟁 블록만 실행하면 되므로 효율적

주요 메시지 타입 요약

L1 (ophost) 메시지

- `MsgRecordBatch` : 배치 제출 (BatchSubmitter용)
- `MsgCreateBridge` : 브리지 생성
- `MsgInitiateTokenDeposit` : 입금 시작 (사용자용)
- `MsgFinalizeTokenWithdrawal` : 출금 완료 (사용자용)
- `MsgProposeOutput` : Output 제안 (Proposer용)
- `MsgDeleteOutput` : Output 삭제 (Challenger용)

L2 (opchild) 메시지

- `MsgExecuteMessages` : 권한 메시지 실행 (BridgeExecutor용)
- `MsgFinalizeTokenDeposit` : 입금 완료 (BridgeExecutor용)
- `MsgInitiateTokenWithdrawal` : 출금 시작 (사용자용)
- `MsgAddValidator` / `MsgRemoveValidator` : 검증자 관리
- `MsgSetBridgeInfo` : 브리지 정보 설정

보안 고려사항

1. 권한 기반 접근 제어

- 각 모듈은 특정 권한을 가진 주소만 중요한 작업 수행 가능
- Authority 주소를 통한 거버넌스 기반 관리

2. 시간 기반 보안

- Finalization Period를 통한 출금 지연으로 챌린지 시간 확보
- Submission Interval을 통한 정기적인 상태 업데이트

3. 암호학적 보안

- Merkle Tree를 통한 출금 증명
- Output Root를 통한 L2 상태 무결성 보장

이 아키텍처는 OPinit의 복잡한 시스템을 체계적으로 구성하여 안전하고 효율적인 Optimistic Rollup 솔루션을 제공합니다.